CODEALPHA INTERNSHIP REPORT

TASK 3: SECURE CODING REVIEW

Internship Domain: Cyber Security
Programming Language: Python
Project Title: Secure Coding Review of a Login System
Internship Organization: CodeAlpha
Intern Name: Charan M R

---

1. INTRODUCTION

Secure coding is the practice of writing software that protects applications and user information from security vulnerabilities. Security flaws in applications can lead to unauthorized access, data theft, and other cyber threats.

As part of the CodeAlpha internship, Task 3 involved reviewing a Python-based login system to identify security vulnerabilities, analyze the source code, and recommend appropriate security improvements.

The project involved examining vulnerable and secure implementations of a login system and using security tools such as Bandit and Pylint to support the review.

2. OBJECTIVES

The main objectives of this project were:

1. To understand common security vulnerabilities in Python applications.
2. To review source code and identify insecure coding practices.
3. To understand SQL injection and its impact on login systems.
4. To use static analysis tools to inspect Python source code.
5. To improve password handling using secure practices and bcrypt.
6. To recommend remediation techniques and document the findings.

3. TOOLS AND TECHNOLOGIES USED

Python: Used to develop and review the login system.

Bandit: A Python security analysis tool used to identify common security issues in source code.

Pylint: A static code analysis tool used to identify programming errors, coding issues, and maintainability problems.

bcrypt: A password-hashing library used to securely hash passwords instead of storing them as plain text.

SQLite: A lightweight database used to demonstrate login-related database operations and SQL injection risks.

Visual Studio Code / Terminal: Used for writing, executing, and testing the Python programs.

4. PROJECT DESCRIPTION

The project focused on reviewing a login system that accepts a username and password and checks the credentials against stored information.

Two implementations were considered:

4.1 Vulnerable Implementation

The vulnerable version demonstrated insecure coding practices, including unsafe construction of SQL queries using user input. This approach can expose an application to SQL injection attacks.

An attacker may manipulate input fields to alter the intended database query, potentially bypassing authentication or accessing unauthorized information.

4.2 Secure Implementation

The secure version focused on improving the application's security by following secure coding practices. These included using parameterized SQL queries, improving password handling, and validating user input appropriately.

The secure implementation was also checked using static analysis tools to identify potential issues.

5. METHODOLOGY

The project was carried out through the following steps:

1. **Environment setup:** Created a dedicated project directory and configured a Python virtual environment.
2. **Dependency installation:** Installed the required packages, including Bandit, Pylint, and bcrypt.
3. **Source code review:** Examined the login application's code to identify potentially unsafe practices.
4. **Vulnerability analysis:** Reviewed the vulnerable implementation for SQL injection and other security risks.

5. **Code remediation:** Applied secure coding principles to improve database queries and password handling.
6. **Static analysis:** Executed Bandit and Pylint against the secure implementation.
7. **Testing and documentation:** Ran the login program and recorded the available tool results.

6. SECURITY VULNERABILITIES IDENTIFIED

6.1 SQL Injection

Description: SQL injection occurs when untrusted input is incorporated into an SQL statement without proper parameterization.

Impact: An attacker may manipulate database queries and potentially bypass authentication or access information without authorization.

Remediation: Use parameterized queries instead of concatenating user input into SQL statements.

6.2 Insecure Password Storage

Description: Storing passwords as plain text exposes credentials if the database is compromised.

Impact: Attackers who obtain the database may gain direct access to user passwords.

Remediation: Store passwords using a dedicated password-hashing algorithm such as bcrypt, with an appropriate work factor. Verify entered passwords using the library's password verification function.

6.3 Insufficient Input Validation

Description: Applications that do not validate inputs appropriately may accept unexpected or malformed data.

Impact: This can increase the risk of application errors and contribute to security weaknesses.

Remediation: Validate input types, lengths, and permitted formats on the server side. Do not rely on input validation as a substitute for parameterized SQL queries.

6.4 Error Handling and Information Disclosure

Description: Detailed database or internal error messages can reveal implementation details.

Impact: Attackers may use these details to understand the application and plan further attacks.

Remediation: Display generic error messages to users while recording appropriate diagnostic details securely in application logs.

Note: These are the security risks reviewed as part of the project. The exact presence of each vulnerability should be confirmed against the final source code.

7. SECURITY IMPROVEMENTS

The following secure coding practices were considered for the secure implementation:

- Use parameterized SQL queries for database operations.
- Hash passwords with bcrypt instead of storing plain-text passwords.
- Verify passwords using secure password-verification functions.
- Validate and sanitize inputs where appropriate.
- Avoid exposing internal errors and sensitive information.
- Apply the principle of least privilege to database access.
- Run static analysis tools regularly during development.
- Test authentication behavior with valid and invalid credentials.

8. STATIC CODE ANALYSIS

8.1 Bandit Analysis

Bandit was used to scan the secure Python implementation for common security issues.

Command executed:

bandit -r secure -f txt -o reports\secure_bandit_report.txt

Observed result: No issues were identified by Bandit during the recorded scan.

This indicates that Bandit did not detect any of the security issues covered by its checks in the scanned code. It does not guarantee that the application is completely secure.

8.2 Pylint Analysis

Pylint was used to assess code quality and identify potential programming and maintainability issues.

Command executed:

pylint secure\login.py > reports\secure_pylint_report.txt

Recorded result:

- Earlier code quality score: 8.08/10
- Improved code quality score: 10.00/10
- Improvement: 1.92 points

The final score indicates that Pylint found no issues affecting its score in the reviewed version. Pylint is primarily a code-quality tool and does not establish that the application is free of security vulnerabilities.

9. TESTING

The login program was executed from the terminal to check its basic authentication behavior.

The following test cases are recommended for documenting the final version:

Test case	Expected behavior
Correct username and password	Login succeeds
Incorrect password	Login is rejected
Unknown username	Login is rejected
Empty username or password	Input is rejected appropriately
SQL injection test input	Authentication is not bypassed
Incorrect password against a stored hash	Password verification fails

The SQL injection test should be performed only in the local test environment. Actual test results should be recorded after executing each case.

10. RESULTS AND DISCUSSION

The secure coding review helped demonstrate the importance of reviewing source code for security weaknesses before deployment.

The recorded static analysis results were:

- Bandit reported no issues in the scanned secure implementation.
- Pylint's code quality score improved from 8.08/10 to 10.00/10.
- The project provided practical experience with Python security analysis and secure authentication design.
- The review highlighted the importance of parameterized SQL queries and appropriate password hashing.

Static analysis results should be combined with manual code review and functional security testing because no single tool can identify every possible vulnerability.

11. CHALLENGES FACED

The main challenges associated with the project included:

1. Understanding how SQL injection can affect authentication systems.
2. Distinguishing security analysis from general code-quality analysis.
3. Configuring the Python environment and installing the required packages.
4. Interpreting the output generated by Bandit and Pylint.
5. Applying secure coding practices while maintaining the expected login functionality.

Working through these challenges improved practical understanding of application security and code review techniques.

12. LEARNING OUTCOMES

Through this task, I gained knowledge of:

- Common security vulnerabilities in Python applications.
- SQL injection risks and their prevention.
- Secure password storage using password hashing.
- Static code analysis using Bandit and Pylint.
- The importance of input validation and error handling.
- The role of manual review and testing in secure software development.

13. CONCLUSION

The CodeAlpha Secure Coding Review task provided practical experience in identifying potential security weaknesses and understanding how secure coding practices improve application reliability.

The project demonstrated the importance of parameterized SQL queries, secure password handling, input validation, and appropriate error management. Bandit and Pylint were used to review the secure implementation, with no issues reported by Bandit and a recorded Pylint score of 10.00/10.

Overall, the task strengthened my understanding of Python application security and the importance of incorporating security checks throughout the software development process.

14. REFERENCES

1. Python Documentation — https://docs.python.org/3/
2. Bandit Documentation — https://bandit.readthedocs.io/
3. Pylint Documentation — https://pylint.readthedocs.io/
4. bcrypt Documentation — https://pypi.org/project/bcrypt/
5. OWASP SQL Injection Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
6. OWASP Password Storage Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

---

Submitted by: Charan M R
Internship Organization: CodeAlpha
Project: Task 3 – Secure Coding Review